SOFTWARE REQUIREMENTS SPECIFICATION
Virtual Pet Simulator

Manage Pet Status (Hunger & Happiness) using the Observer Pattern

OOSE / UML Project Document

Exam-ready SRS with six UML diagrams

1. Introduction

Purpose: The Virtual Pet Simulator lets a user care for a virtual pet. The system maintains hunger and happiness, changes them through actions and periodic updates, and automatically refreshes status displays using the Observer pattern.

Scope: Create/select a pet, view status, feed, play, receive automatic status updates, observe changes, and save/load pet data.

Actors: Pet Owner and System Timer.

2. Functional Requirements

ID	Requirement
FR-01	Create or load a virtual pet.
FR-02	Display hunger, happiness and overall condition.
FR-03	Allow the user to feed the pet.
FR-04	Allow the user to play with the pet.
FR-05	Update pet status periodically using a timer.
FR-06	Notify registered observers whenever relevant state changes.
FR-07	Refresh the status display after notification.
FR-08	Save and load pet data.
FR-09	Keep status values within the valid range.
FR-10	Derive a condition such as Happy, Normal, Hungry or Critical.

3. Non-Functional Requirements

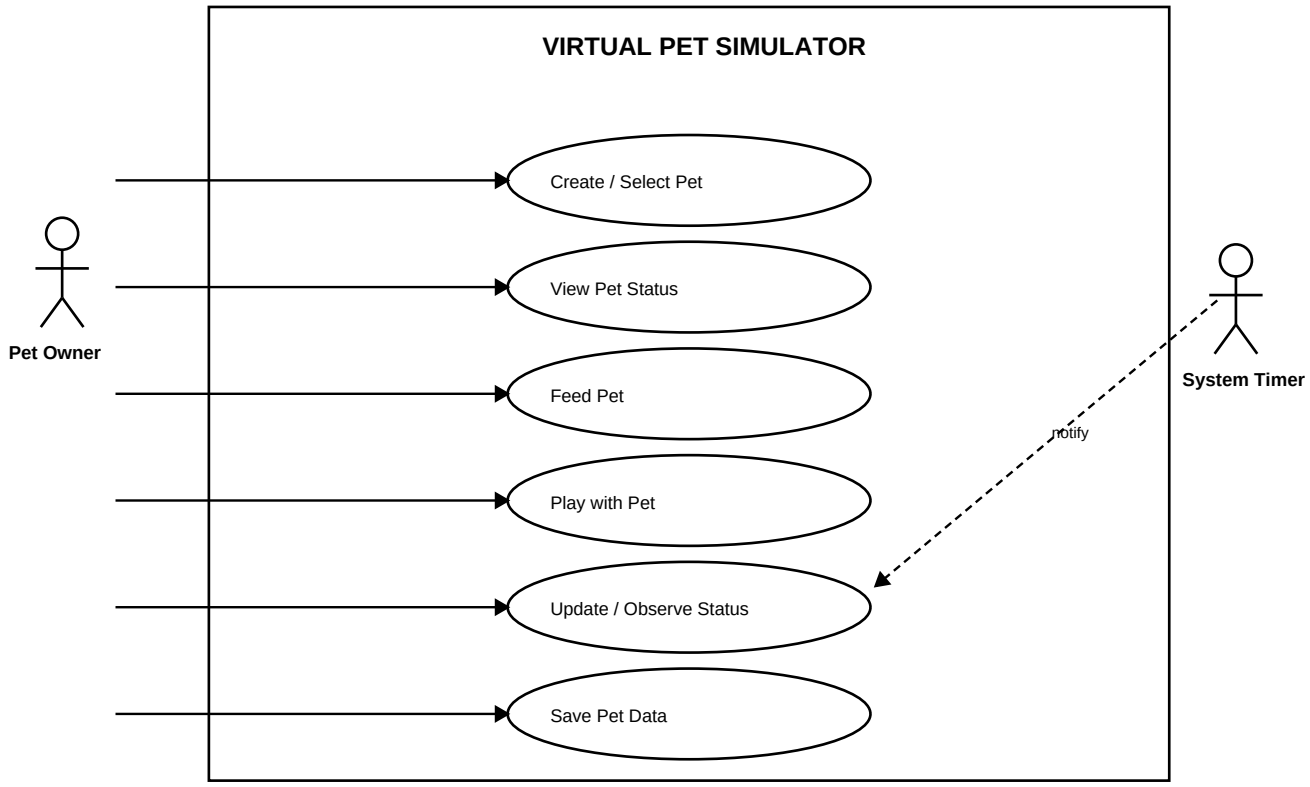
Category	Requirement
Usability	Simple interface with clear status and actions.
Performance	User actions and notifications should update quickly.
Reliability	Pet values and saved data should remain consistent.
Maintainability	Pet, UI, observer and persistence logic should be modular.
Scalability	Additional observers and pet types should be easy to add.
Security	Persisted data should be protected from accidental corruption.

4. Observer Pattern

Subject: Pet. It owns hunger/happiness state and a list of observers. **Observer:** PetObserver interface. **Concrete Observer:** StatusDisplay. After feed(), play(), or a timer update changes the pet, Pet calls notifyObservers(), which invokes update(pet) on registered observers. This keeps displays synchronized without tightly coupling Pet to a particular UI.

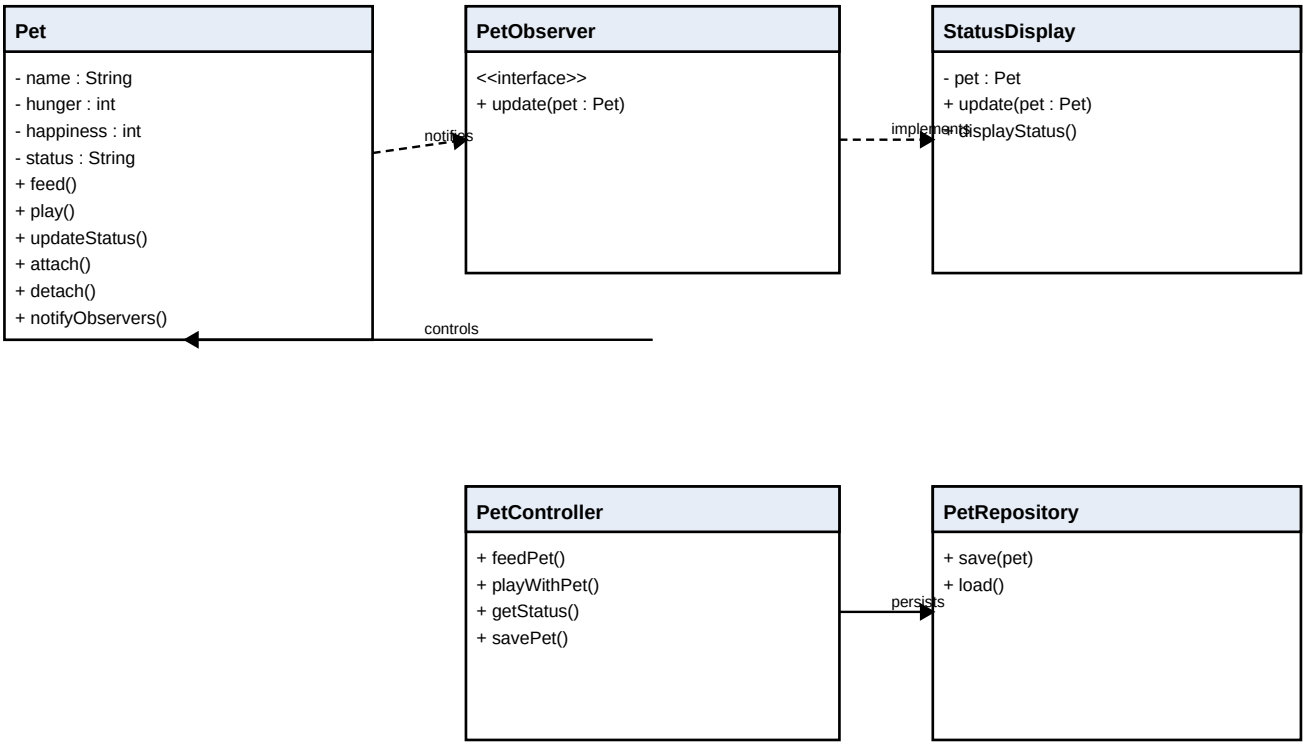
5. UML Diagram 1 — Use Case Diagram

Shows who interacts with the system and the major functions.



6. UML Diagram 2 — Class Diagram

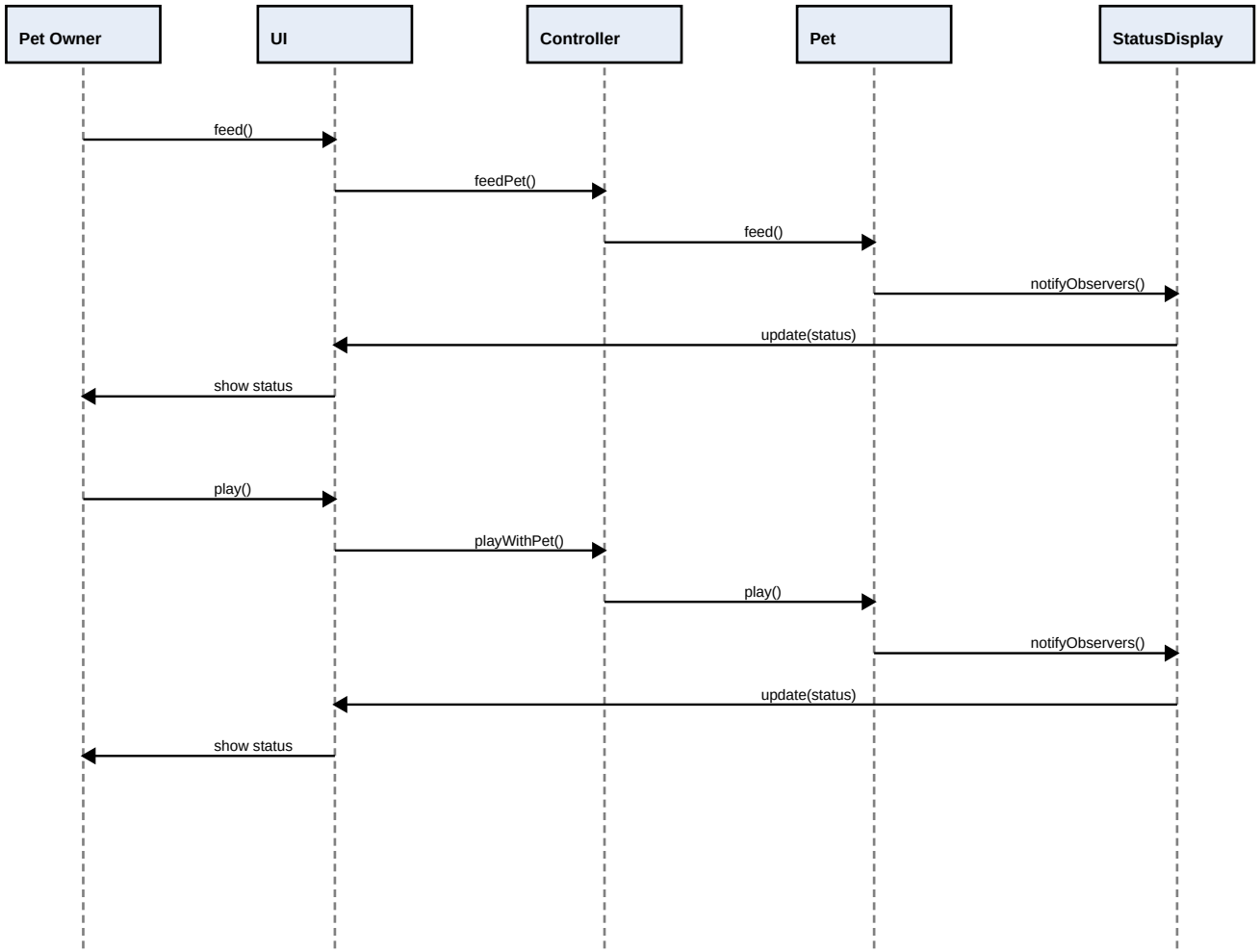
Shows classes, responsibilities and the Observer relationship.



Observer pattern: Pet = Subject; PetObserver = Observer interface; StatusDisplay = Concrete Observer.

7. UML Diagram 3 — Sequence Diagram

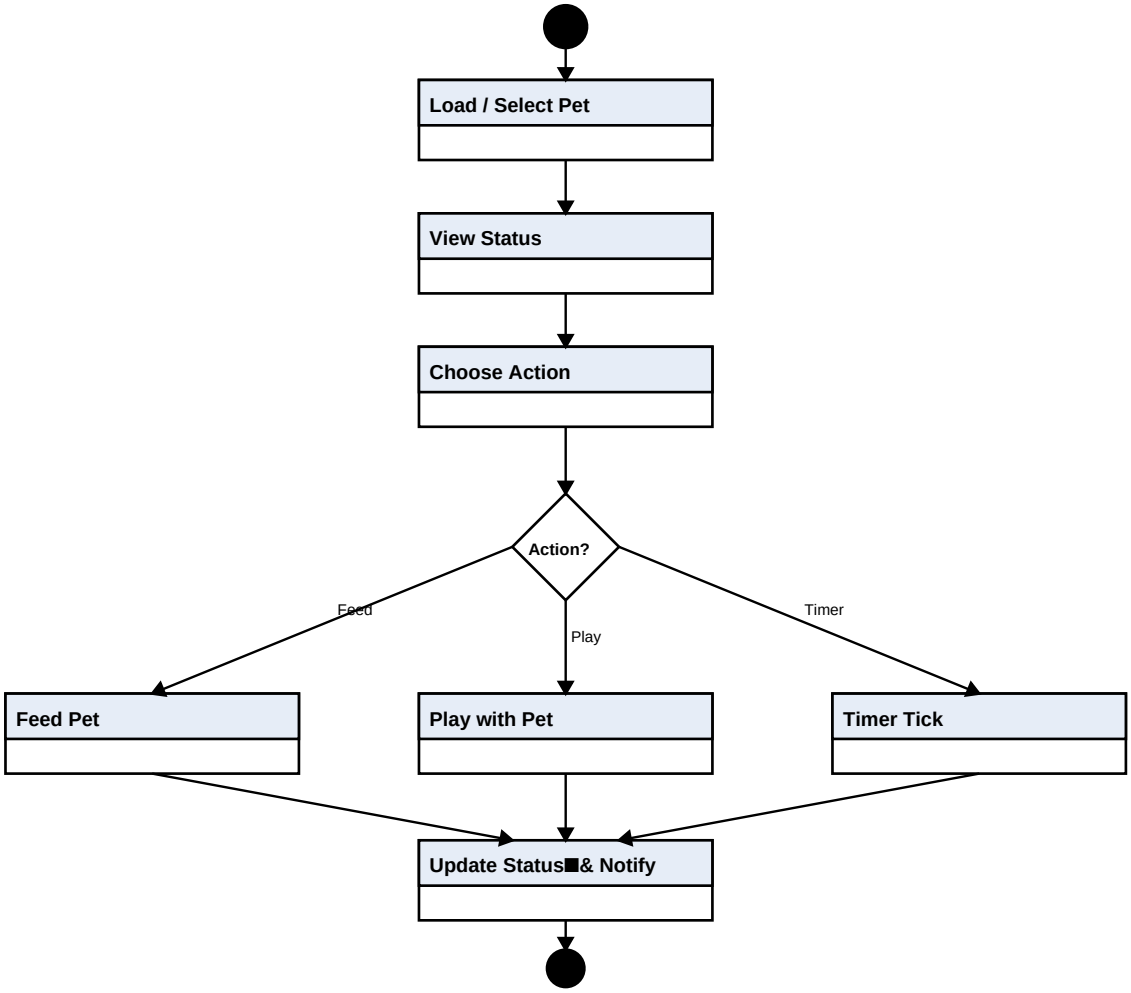
Shows the order of messages during feed/play operations.



User action -> Controller -> Pet -> Observer notification -> UI refresh.

8. UML Diagram 4 — Activity Diagram

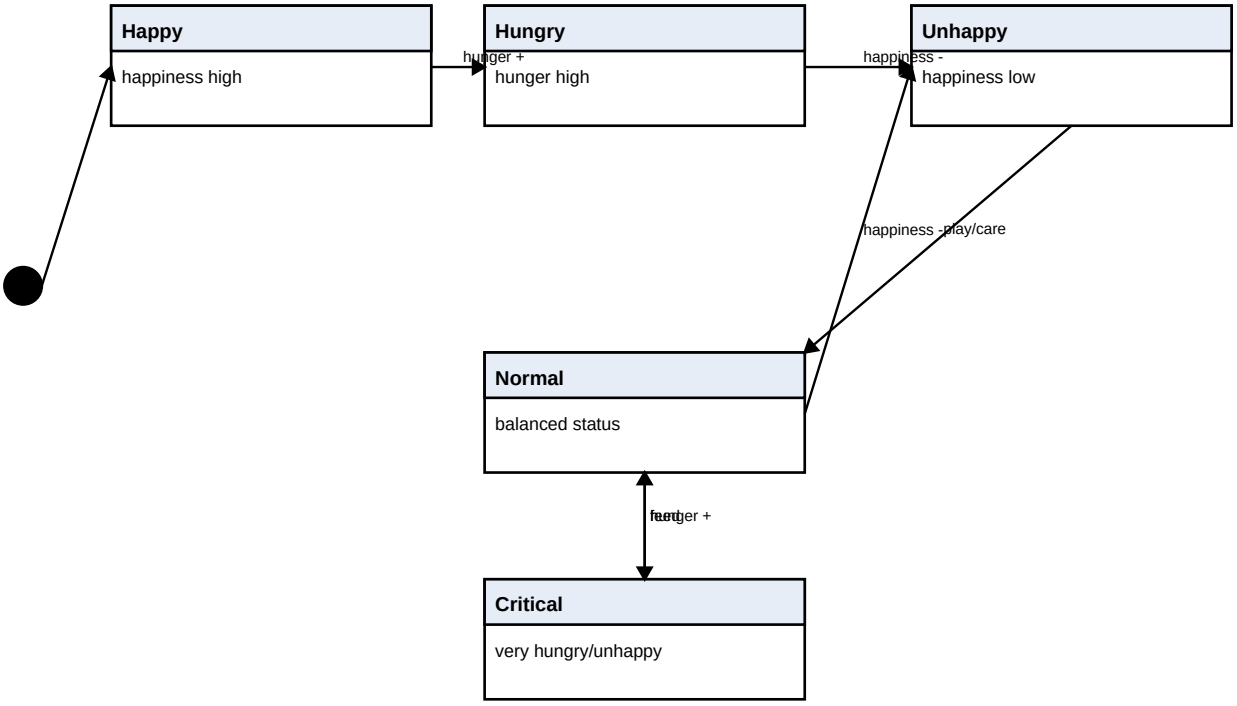
Shows the workflow and decision points.



9. UML Diagram 5 — State Diagram

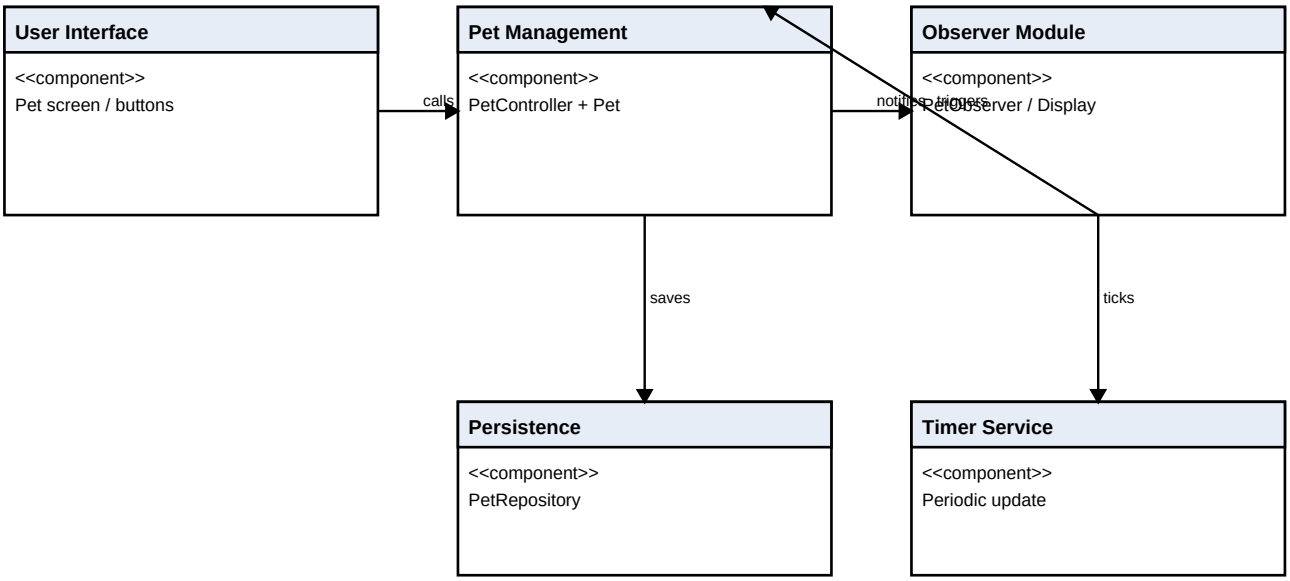
Shows changes in the pet's overall condition.

Pet condition changes as hunger and happiness values change.



10. UML Diagram 6 — Component Diagram

Shows the major software modules and their dependencies.



11. Use Case Summary

Use Case	Actor	Brief Scenario
Create / Select Pet	Pet Owner	Create a new pet or load an existing pet.
View Pet Status	Pet Owner	See hunger, happiness and condition.
Feed Pet	Pet Owner	Feed the pet; hunger improves and observers are notified.
Play with Pet	Pet Owner	Play; happiness improves and observers are notified.
Automatic Status Update	System Timer	Timer changes status periodically and triggers notification.
Save Pet Data	Pet Owner / System	Store current pet state.

12. Conclusion

The Virtual Pet Simulator demonstrates OOSE modeling and the Observer design pattern. The six UML diagrams provide complementary views: use cases show system functions, the class diagram shows structure, the sequence diagram shows interactions, the activity diagram shows workflow, the state diagram shows status changes, and the component diagram shows the software architecture.